

Instituto Tecnológico y de Estudios Superiores de Monterrey

Campus Estado de México

Momento de Retroalimentación: Módulo 2 Uso de framework o biblioteca de aprendizaje máquina para la implementación de una solución.

Jorge Adolfo Ramírez Uresti

Israel González Huerta A01751433

Inteligencia artificial avanzada para la ciencia de datos (Gpo 601)

Miércoles 9 de septiembre de 2026

Introducción

Implementé el algoritmo de Random Forest con SciKit-learn con el Titanic Dataset de Kaggle (obtenido de <https://www.kaggle.com/datasets/yasserh/titanic-dataset?select=Titanic-Dataset.csv>). Decidí implementar Random Forest porque conozco la teoría detrás de su funcionamiento y porque para la otra entrega implementé un árbol de decisión por mi propia cuenta, pero limitado en la cantidad de hiperparámetros que puedo modificar, quise usar este modelo para comprender los efectos de los hiperparámetros en el desempeño. El dataset de Titanic fue elegido porque ofrece una combinación de variables numéricas y categóricas.

Dataset

Del dataset en el sitio de Kaggle están disponibles las columnas de PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin y Embarked. Se extrajeron todas las columnas excepto por Cabin porque el 77% de sus valores eran nulos y Embarked porque consideré que sus valores no aportarían información relevante para la predicción.

De las columnas seleccionadas, dentro del programa se omiten las columnas de PassengerID, Name y Ticket por consideración de que no aportan información para determinar si alguien sobrevivió o no.

Se usó una división donde 80% de los datos (seleccionados de manera pseudoaleatoria) son usados para entrenamiento y 20% para evaluación; hay 712 datos de entrenamiento y 179 de prueba.

Implementación del Random Forest

La implementación puede trabajar con cualquier dataset, pero se tiene que modificar el código para indicar cuál es el nombre del archivo .csv, las columnas categóricas, las columnas numéricas y la columna target. Los hiperparámetros que se pueden modificar son la cantidad de árboles que conforman el forest, la profundidad máxima de los árboles, el mínimo de muestras por división, mínimo de muestras por hoja, número máximo de características y la semilla utilizada para controlar la generación de números pseudoaleatorios y permitir la reproducibilidad de los resultados.

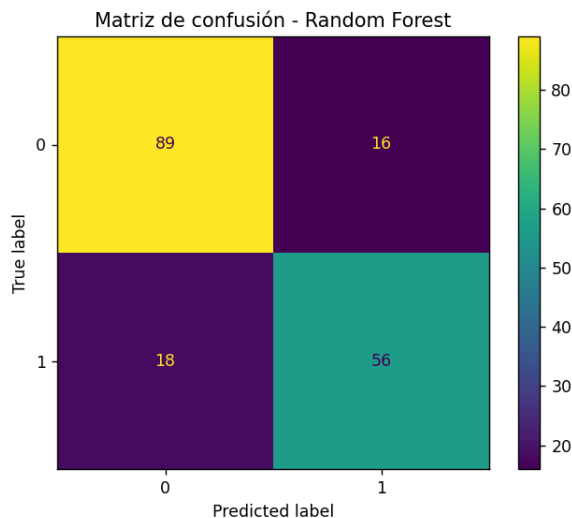
Resultados

En esta sección incluiré resultados de correr el árbol con distintos valores en los hiperparámetros, qué significan y cómo se comparan será discutido en la siguiente sección “Análisis”. Solo incluiré imágenes de una vez que haya corrido el árbol (con sus respectivos cambios) que me parezca representativo del cambio con el fin de evitar imágenes que solo ilustren la misma conducta que iba a discutir en la sección de análisis.

Si hago referencia valores “originales” o “base” estoy hablando de los valores incluidos en esta tabla para los resultados 1. Valores originales.

1. Valores originales

Hiperparámetro	Valor
Número de árboles	100
Profundidad máxima	None
Mínimo de muestras por división	2
Mínimo de muestras por hoja	1
Máximo de características	“sqrt”



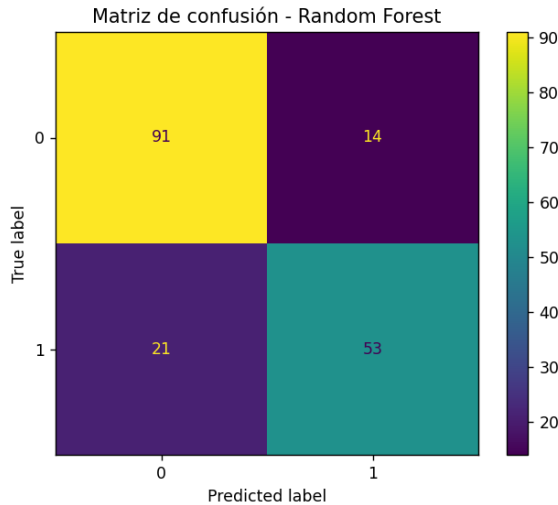
```
=====
ESTADÍSTICAS DEL MODELO
=====
Accuracy de entrenamiento: 97.89%
Accuracy de prueba:      81.01%
Diferencia:              16.89%

=====
REPORTE DE CLASIFICACIÓN
=====
```

	precision	recall	f1-score	support
0	0.83	0.85	0.84	105
1	0.78	0.76	0.77	74
accuracy			0.81	179
macro avg	0.80	0.80	0.80	179
weighted avg	0.81	0.81	0.81	179

2. Modificación al número de árboles

Manteniendo todos los hiperparámetros iguales y bajando el número de árboles a 10. Se ven pequeños decrementos en la mayoría de las métricas.

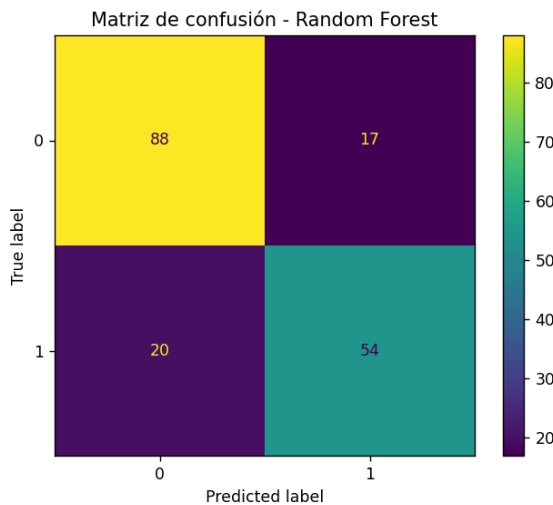


```
=====
ESTADÍSTICAS DEL MODELO
=====
Accuracy de entrenamiento: 95.51%
Accuracy de prueba:      80.45%
Diferencia:              15.06%

=====
REPORTE DE CLASIFICACIÓN
=====
```

	precision	recall	f1-score	support
0	0.81	0.87	0.84	105
1	0.79	0.72	0.75	74
accuracy			0.80	179
macro avg	0.80	0.79	0.80	179
weighted avg	0.80	0.80	0.80	179

Incrementando la cantidad de árboles a 1000 se ven pequeños decrementos en la mayoría de las métricas.



```
=====
ESTADÍSTICAS DEL MODELO
=====
Accuracy de entrenamiento: 97.89%
Accuracy de prueba:      79.33%
Diferencia:              18.56%

=====
REPORTE DE CLASIFICACIÓN
=====
```

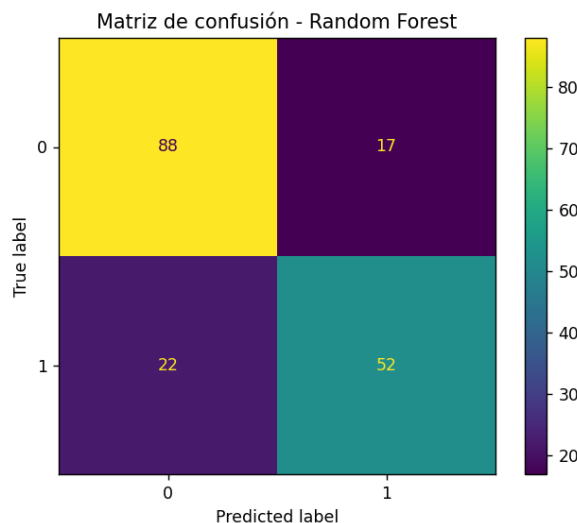
	precision	recall	f1-score	support
0	0.81	0.84	0.83	105
1	0.76	0.73	0.74	74
accuracy			0.79	179
macro avg	0.79	0.78	0.79	179
weighted avg	0.79	0.79	0.79	179

3. Limitar la profundidad máxima

Dado que los valores originales no tienen un límite de profundidad decidí probar iniciando con profundidad de 1 y aumentando para ver como las métricas de acercan hacia los valores cuando no hay límite.

Accuracy con los datos de entrenamiento inicia alrededor de 78.8% y crece conforme el límite de profundidad hasta que llega a un valor alrededor de 97.9%. Accuracy con los valores de prueba inicia alrededor de 78.2% y deja de crecer cerca del 81%. Notoriamente, los valores de accuracy con los datos de entrenamiento siguen incrementando después de que el accuracy con los datos de prueba ha dejado de crecer, esto es un indicador rudimentario de overfitting (aunque usualmente esto va a acompañado en disminuciones en el accuracy para los valores de prueba).

Resultados con profundidad máxima de 1:

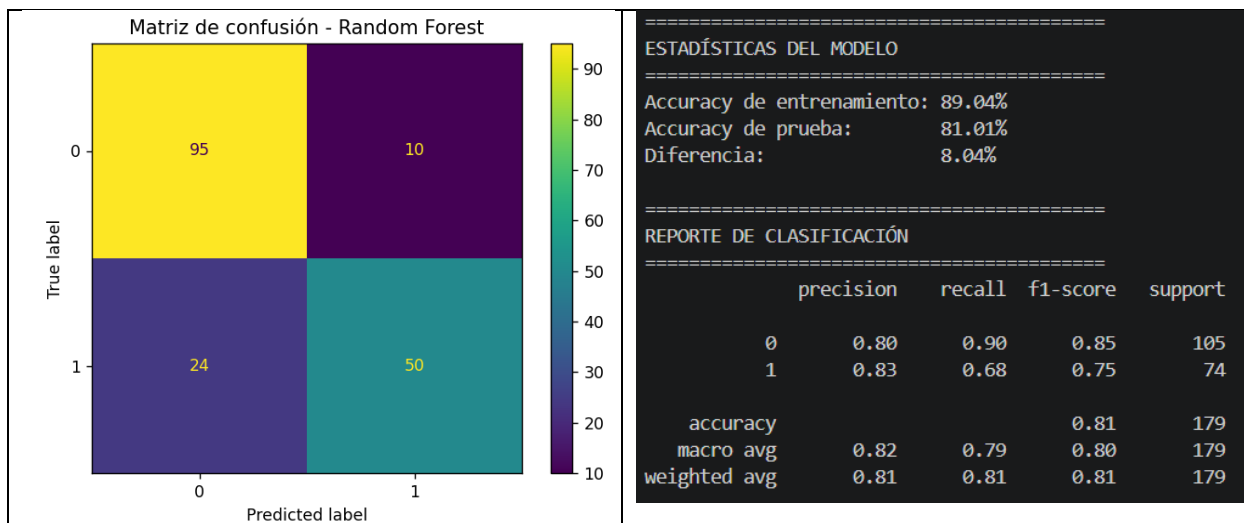


```
=====
ESTADÍSTICAS DEL MODELO
=====
Accuracy de entrenamiento: 78.79%
Accuracy de prueba:       78.21%
Diferencia:               0.58%

=====
REPORTE DE CLASIFICACIÓN
=====
```

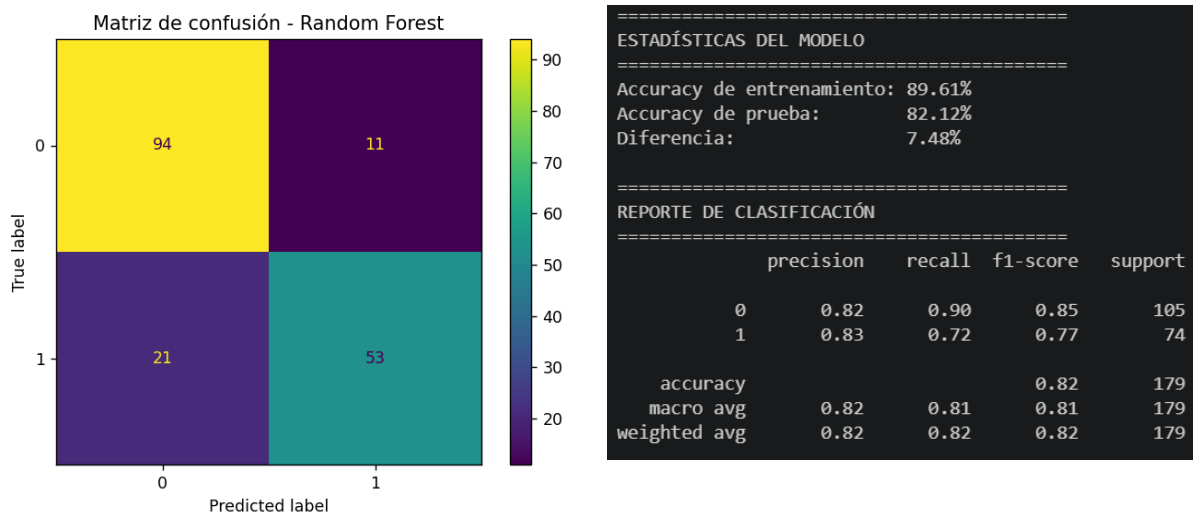
	precision	recall	f1-score	support
0	0.80	0.84	0.82	105
1	0.75	0.70	0.73	74
accuracy			0.78	179
macro avg	0.78	0.77	0.77	179
weighted avg	0.78	0.78	0.78	179

Resultados con profundidad máxima de 7, este es el punto encontrado en el que el accuracy de prueba deja de incrementar.



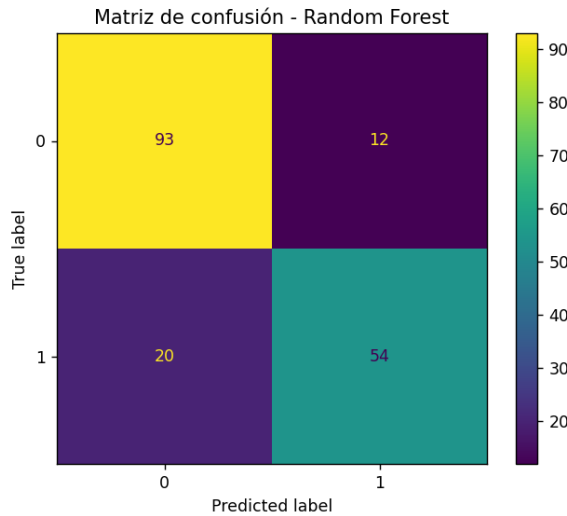
4. Mínimo de muestras por división

El accuracy de entrenamiento disminuye (acercándose al valor del accuracy de prueba), mientras que el accuracy de prueba sí incrementó hasta llegar a un valor de 82.2%. En cierto punto este modelo supera al original en las métricas, encontré que el valor que da mejor resultados es 12.



5. Mínimo de muestras por hoja

Los resultados mejoraron con un valor de 2 y 3, pero después de ese punto solo vi pérdidas en rendimiento.



```

=====
ESTADÍSTICAS DEL MODELO
=====
Accuracy de entrenamiento: 89.19%
Accuracy de prueba:      82.12%
Diferencia:              7.06%

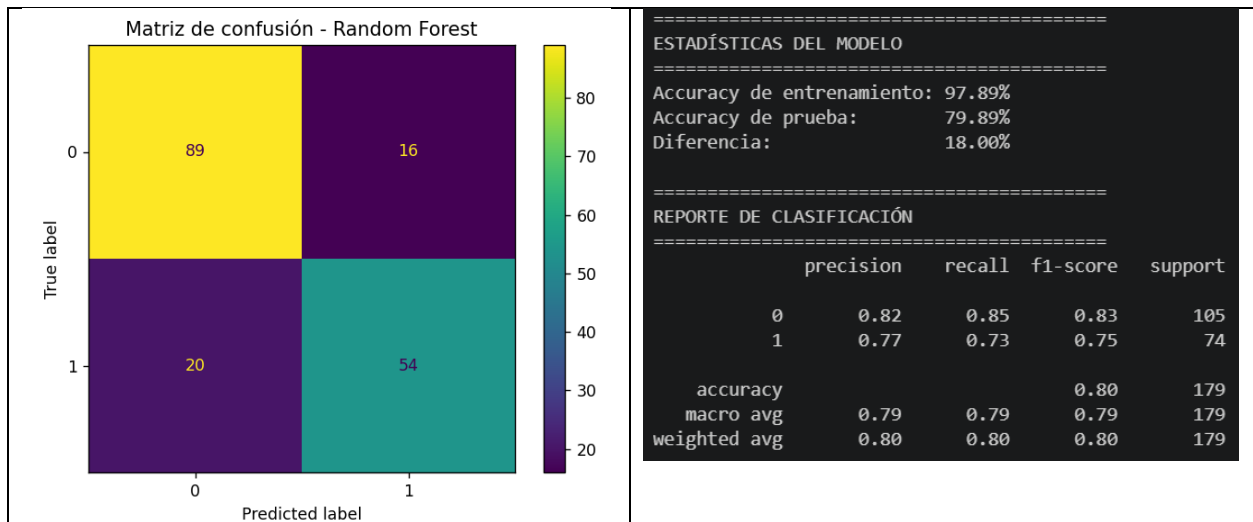
=====
REPORTE DE CLASIFICACIÓN
=====

```

	precision	recall	f1-score	support
0	0.82	0.89	0.85	105
1	0.82	0.73	0.77	74
accuracy			0.82	179
macro avg	0.82	0.81	0.81	179
weighted avg	0.82	0.82	0.82	179

6. Número máximo de características consideradas en cada división

El accuracy de prueba se mantuvo igual para todos los valores excepto para 5 y 6, para los cuáles bajó el accuracy y las otras métricas. Aquí muestro los resultados de dejar el máximo de características en 5.



Análisis

Los experimentos realizados muestran que los hiperparámetros del Random Forest tienen efectos diferentes sobre el desempeño del modelo. En general, los cambios que limitan la complejidad de los árboles tuvieron un mayor impacto que los cambios en la cantidad de árboles. Esto puede observarse principalmente en los experimentos realizados con la profundidad máxima, el mínimo de muestras por división y el mínimo de muestras por hoja.

Cantidad de árboles

Cambiar la cantidad de árboles de 100 a 10 y 1000 produjo cambios pequeños en las métricas. Esto indica que, para este dataset, aumentar considerablemente la cantidad de árboles no representó una mejora significativa. Por lo tanto, 100 árboles ofrece un equilibrio razonable entre desempeño y costo computacional. Los otros hiperparámetros me parece que tienen un impacto más grande en el desempeño.

Profundidad máxima

Al aumentar la profundidad, el accuracy de entrenamiento continuó creciendo, mientras que el de prueba dejó de mejorar aproximadamente en una profundidad de 7. Esto indica que permitir árboles más complejos mejora el aprendizaje de los datos de entrenamiento, pero no necesariamente la capacidad de generalización. La diferencia entre ambos desempeños es una señal de posible sobreajuste.

Mínimo de muestras por división

Aumentar `min_samples_split` redujo el accuracy de entrenamiento, pero mejoró el de prueba hasta alcanzar aproximadamente 82.2% con un valor de 12. Esto indica que limitar las divisiones de los árboles ayudó a reducir su complejidad y mejorar su capacidad de generalización.

Mínimo de características

Los valores de 2 y 3 mejoraron el desempeño respecto al valor base de 1. Sin embargo, valores mayores comenzaron a reducir las métricas. Esto muestra que limitar ligeramente

el tamaño de las hojas puede ayudar a reducir el sobreajuste, pero hacerlo demasiado puede limitar la capacidad del modelo.

Máximo de características

Este hiperparámetro tuvo un efecto menor. La mayoría de los valores produjeron resultados similares, mientras que utilizar 5 o 6 características redujo el desempeño. Mantener `max_features = "sqrt"` permitió conservar la aleatoriedad característica del Random Forest sin afectar negativamente las métricas.

Comparación general

Los resultados indican que controlar la complejidad de los árboles tuvo mayor impacto que modificar la cantidad de árboles o características. En particular, `min_samples_split = 12` produjo una mejora notable en el accuracy de prueba, mientras que `min_samples_leaf` también mostró mejoras con valores pequeños. Esto demuestra la importancia de ajustar los hiperparámetros de acuerdo con el comportamiento del modelo sobre datos de prueba.

Conclusión

La implementación de Random Forest con Scikit-learn permitió comprobar que los hiperparámetros tienen un efecto importante en el desempeño del modelo, especialmente aquellos relacionados con la complejidad de los árboles. Los experimentos mostraron que limitar esta complejidad puede mejorar la capacidad de generalización y reducir el sobreajuste.

El mejor resultado observado al modificar individualmente los hiperparámetros se obtuvo con `min_samples_split = 12`, alcanzando un accuracy de prueba de aproximadamente 82.2%. En cambio, aumentar considerablemente la cantidad de árboles tuvo un efecto mucho menor sobre las métricas.

En general, la implementación permitió no solo utilizar Random Forest mediante una biblioteca de aprendizaje automático, sino también comprender de manera práctica cómo la configuración del modelo influye en sus resultados y en su capacidad para realizar predicciones sobre datos que no utilizó durante el entrenamiento.